

FIFA World Cup 2026

Prediction System

Code Walkthrough & ML Methods Guide

Ensemble Model (XGBoost + Logistic Regression + Random Forest)

Dixon-Coles Goal Prediction Model

Multi-Layer Architecture with Live Adjustments

Validation: 59.4% accuracy on 2022 World Cup

1. System Architecture

The system has two main components: a training pipeline (run once) that produces a model file, and a Streamlit web app that uses the model for live predictions. The prediction engine uses a 3-layer architecture that combines historical statistical models with real-time adjustments.

Training Pipeline:

```
international_results.csv (49,477 matches)
-> builder.py (single-pass feature extraction)
-> ensemble.py (XGBoost + LR + RF classifier)
-> dixon-coles.py (MLE goal model)
-> model.pkl (saved to disk)
```

App Runtime (per page render):

```
User input (scores, squad) -> session_state -> JSON files
-> predictions.py loads model.pkl
-> Layer 1: Ensemble + Dixon-Coles (base prediction)
-> Layer 2: Tournament adjustments (rest days, pedigree)
-> Layer 3: Live adjustments (squad, form)
-> Render predictions in UI
```

2. Elo Rating System

What is Elo?

Elo is a rating system originally designed for chess (Arpad Elo, 1960). Each team starts at 1500. After each match, ratings are updated based on the result vs expectation. If a low-rated team beats a high-rated team, the rating transfer is large. If the favorite wins as expected, the transfer is small.

The Math

Expected score for home team:

$$E_{\text{home}} = 1 / (1 + 10^{((\text{Elo}_{\text{away}} - \text{Elo}_{\text{home}} - \text{HomeAdv}) / 400)})$$

After the match:

$$\text{New_Elo} = \text{Old_Elo} + K * \text{GD_mult} * (\text{Actual} - \text{Expected})$$

Where:

$K = 60$ (World Cup), 50 (continental), 40 (qualifiers), 20 (friendlies)

$\text{GD_mult} = 1.0$ (1 goal), 1.5 (2 goals), $(11 + \text{GD}) / 8$ (3+ goals)

$\text{HomeAdv} = 100$ (non-neutral), 0 (neutral/World Cup)

$\text{Actual} = 1.0$ (win), 0.5 (draw), 0.0 (loss)

Why it works for football

Elo difference is the single strongest predictor of match outcomes (~60% accuracy alone). It captures long-term team strength that adapts to results. Our implementation uses variable K-factors (World Cup results matter 3x more than friendlies) and goal difference multipliers (winning 5-0 gives more rating boost than winning 1-0).

References

World Football Elo Ratings

https://en.wikipedia.org/wiki/World_Football_Elo_Ratings

Wikipedia article explaining the football-specific Elo implementation used by eloratings.net.

Elo Rating System (original)

https://en.wikipedia.org/wiki/Elo_rating_system

The mathematical foundation of Elo ratings, expected scores, and K-factor selection.

3. Feature Engineering

Single-Pass Feature Builder

We process all 49,477 matches in chronological order in a single pass. For each match, we snapshot the current state BEFORE updating it. This ensures no data leakage - each match's features only reflect information available at that point in time.

The 14 Features

Team Strength (3 features):

- elo_home - Current Elo rating of home team
- elo_away - Current Elo rating of away team
- elo_diff - Difference (most important single feature)

Recent Form (8 features):

- form_home_pts_5 - Home team's avg points/match (last 5)
- form_away_pts_5 - Away team's avg points/match (last 5)
- form_home_pts_10 - Home team's avg points/match (last 10)
- form_away_pts_10 - Away team's avg points/match (last 10)
- form_home_gf_5 - Home team's avg goals scored (last 5)
- form_away_gf_5 - Away team's avg goals scored (last 5)
- form_home_gc_5 - Home team's avg goals conceded (last 5)
- form_away_gc_5 - Away team's avg goals conceded (last 5)

Head-to-Head (3 features):

- h2h_matches - Number of prior meetings
- h2h_home_win_pct - Historical win percentage
- h2h_goal_diff - Net goals in prior meetings

The builder processes 49k matches in 0.5 seconds by maintaining rolling accumulators (Python lists/dicts) instead of recomputing from scratch for each match.

4. XGBoost Classifier

What is XGBoost?

XGBoost (eXtreme Gradient Boosting) is an ensemble of decision trees trained sequentially. Each new tree corrects the errors of the previous trees. It's the dominant algorithm in tabular data competitions (Kaggle) due to high accuracy, speed, and built-in regularization.

How it works

1. Start with a base prediction (class probabilities)
2. Compute residuals (errors) for each training sample
3. Fit a new decision tree to predict those residuals
4. Add the new tree's predictions (scaled by `learning_rate`)
5. Repeat for `n_estimators=150` trees

Each tree has:

```
max_depth=5 (limits complexity, prevents overfitting)
subsample=0.8 (80% of data per tree for diversity)
colsample_bytree=0.8 (80% of features per tree)
```

Output: `P(loss)`, `P(draw)`, `P(win)` for each match

Objective: `multi:softprob` (multiclass probability)

Why we use it

XGBoost captures non-linear feature interactions (e.g., high Elo + poor recent form = vulnerable favorite). It handles missing data gracefully and provides feature importance scores. However, it can overfit if not regularized, which is why we ensemble it with simpler models.

References

XGBoost Paper (Chen & Guestrin, 2016)

<https://arxiv.org/abs/1603.02754>

The original paper introducing XGBoost. Covers the algorithm, regularization, and system design.

XGBoost Documentation

<https://xgboost.readthedocs.io/en/latest/>

Official docs with parameters, tutorials, and API reference.

5. Logistic Regression

What is it?

Logistic Regression is a linear classifier that models $P(\text{class}|\text{features})$ using the logistic (sigmoid) function. For multiclass (3 outcomes), it uses the softmax function. Despite its simplicity, it often produces well-calibrated probabilities - meaning when it says 60% chance of home win, it's right roughly 60% of the time.

The Math

```
For 3 classes (loss, draw, win):  
  
score_k = w_k . x + b_k    (linear combination of features)  
  
P(class=k) = exp(score_k) / sum(exp(score_j) for all j)  
  
Training: maximize log-likelihood using L-BFGS optimizer  
Regularization: C=1.0 (L2 penalty to prevent overfitting)
```

Why it's in the ensemble

LR provides a strong linear baseline. In our ensemble, it got the highest weight (56%) because its well-calibrated probabilities are excellent for blending. XGBoost may overfit to specific patterns, but LR's smooth probability surface averages out those errors.

References

Logistic Regression (scikit-learn)

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
scikit-learn documentation covering multinomial logistic regression.

6. Random Forest

What is it?

Random Forest builds many independent decision trees (200 in our case), each trained on a random subset of data and features. Final prediction = average of all trees' votes. This 'wisdom of crowds' approach is robust to outliers and noise.

How it differs from XGBoost

Random Forest:

- Trees are independent (trained in parallel)
- Each tree sees random subset of data (bagging)
- Reduces variance (overfitting)
- Less prone to learning noise

XGBoost:

- Trees are sequential (each corrects previous errors)
- All trees see full data
- Reduces bias (underfitting)
- Can overfit if not regularized

In our ensemble

RF got the lowest weight (12%) - it adds diversity but doesn't dominate because football prediction is fundamentally a probabilistic problem where calibrated probabilities (LR's strength) matter more than raw accuracy.

References

Random Forests (Breiman, 2001)

<https://link.springer.com/article/10.1023/A:1010933404324>

The seminal paper introducing Random Forests.

7. Ensemble Blending

Why blend multiple models?

Each model has different strengths and failure modes. Blending reduces the chance that one model's error dominates the final prediction. This is called 'model stacking' or 'ensemble averaging'.

Weight Optimization

Goal: find weights [w_xgb, w_lr, w_rf] that minimize log-loss

Log-loss = $-\sum(\log(P(\text{correct_class})))$ for all validation matches

Method: Nelder-Mead optimization (derivative-free)

1. Try different weight combinations
2. Measure log-loss on held-out validation data
3. Converge to optimal weights

Result: XGB=0.32, LR=0.56, RF=0.12

(LR dominates because it's best-calibrated)

References

Ensemble Methods in Machine Learning

https://link.springer.com/chapter/10.1007/3-540-45014-9_1

Dietterich (2000) - overview of why ensembles work better than individual models.

Stacking and Blending

<https://mlwave.com/kaggle-ensembling-guide/>

Practical guide to ensemble stacking techniques used in competitions.

8. Dixon-Coles Model

The Problem with Independent Poisson

A basic approach models home goals $\sim \text{Poisson}(\lambda)$ and away goals $\sim \text{Poisson}(\mu)$ independently. But in real football, low-scoring outcomes (0-0, 1-0, 0-1, 1-1) occur more often than independent Poisson predicts. Teams play more cautiously in tight matches.

The Dixon-Coles Correction

```
P(home=x, away=y) = tau(x, y, lambda, mu, rho)
                    * Poisson(x; lambda)
                    * Poisson(y; mu)

tau correction factor (only affects low scores):
tau(0, 0) = 1 - lambda * mu * rho
tau(0, 1) = 1 + lambda * rho
tau(1, 0) = 1 + mu * rho
tau(1, 1) = 1 - rho
tau(x, y) = 1.0 (for all other scores)
```

When $\rho < 0$ (our model: $\rho = -0.17$):

- > 0-0 draws become MORE likely
- > 1-1 draws become MORE likely
- > This matches real football data

Per-Team Parameters

Each team has:

- attack_i - how good at scoring (log-scale)
- defence_i - how bad at preventing goals (log-scale)

For match Home vs Away:

```
lambda = exp(attack_home + defence_away + home_advantage)
mu      = exp(attack_away + defence_home)
```

Fitting: Maximum Likelihood Estimation

- Maximize: $\sum(\text{weight}_i * \log(P(\text{observed_score}_i)))$
- Weights: exponential decay (recent matches count more)
- Optimizer: L-BFGS-B (fast for large parameter spaces)
- Constraint: $\sum(\text{attack}) = 0$ (identifiability)

Why this is the gold standard for football

Dixon-Coles (1997) remains the most-cited paper in football analytics. Its key insight - that goal independence breaks down at low scores - has been validated repeatedly. Our implementation uses time-decay weighting (half-life=365 days) so the model adapts to teams improving or declining over time.

References

Dixon & Coles (1997) - Original Paper

<https://doi.org/10.1111/1467-9876.00065>

"Modelling Association Football Scores and Inefficiencies in the Football Betting Market" - Journal of the Royal Statistical Society. The

FIFA World Cup 2026 Prediction System - Code & ML Walkthrough

foundational paper for football score prediction.

Implementing Dixon-Coles in Python

<https://dashee87.github.io/football/python/predicting-football-results-with-statistical-modelling-dixon-coles/>

Excellent tutorial implementing Dixon-Coles step-by-step with Python code.

Poisson Distribution for Football Goals

https://en.wikipedia.org/wiki/Poisson_distribution

Mathematical background on why goals follow a Poisson process.

9. Multi-Layer Prediction Architecture

Layer 1: Historical Base (Stable)

Combines the ensemble classifier (W/D/L probabilities) with Dixon-Coles (score prediction). The final Layer 1 output blends both 50/50 for win/draw/loss probabilities. Score prediction uses Dixon-Coles exclusively (the ensemble doesn't predict scores).

Layer 2: Tournament Adjustments

Applied on top of Layer 1 to account for World Cup-specific factors:

- WC pedigree: teams with more WC experience get a small boost (2% per level)
- Rest days: <3 days between matches = 8% xG reduction + 4% win probability drop
- All WC matches treated as neutral venue (no home advantage in Dixon-Coles)

Layer 3: Live Adjustments (Real-Time)

Updated every time the page renders based on user input:

- Squad availability: each unavailable player reduces team's xG by 2% (max 30% total)
- In-tournament form: computed from actual scores entered; scales xG up to +/-15%
- This allows predictions to adapt as the tournament progresses

Score Consistency Rule

The 'likely score' shown in the UI must be consistent with the predicted outcome. If the model predicts home win (>50%), we find the most probable HOME WIN scoreline from the Poisson/Dixon-Coles probability grid. This avoids showing '1-1' when the model says 55% home win.

10. Model Validation

2022 World Cup Backtest

We tested the model on all 64 matches of the Qatar 2022 FIFA World Cup (data the model was trained on - but features are computed using only pre-match information).

Results:

Accuracy: 59.4% (38/64 matches predicted correctly)
Random baseline: 33%
Professional bookmakers: ~60%

Breakdown:

Group stage: slightly higher accuracy (more predictable)
Knockout: lower accuracy (upsets more common)

Notable correct predictions:

Argentina beat Australia, France beat Poland,
England beat Senegal, Brazil beat South Korea

Notable misses (upsets):

Saudi Arabia beat Argentina (model gave Argentina 64%)
Morocco beat Spain (model gave Spain 43%)
Japan drew Croatia (model gave Japan 54%)

What limits accuracy?

Football is inherently unpredictable - even the best models max out around 65-70% because:

- Individual moments of brilliance/error (red cards, penalties, goalkeeper mistakes)
- Tactical changes mid-game that statistics can't capture
- Psychological factors (pressure, motivation, fatigue)
- Referee decisions

Our 59.4% is strong for a pure statistical model without betting market data.

11. Potential Improvements

Data that could improve accuracy

Each of these could add 1-5% accuracy:

- Squad market values (Transfermarkt) - correlates with team quality
- Expected Goals (xG) data from StatsBomb/FBref - better than raw goals
- Betting odds integration - market consensus is a powerful signal
- Player-level ratings (FIFA game ratings, WhoScored) - individual quality
- Tactical features (formation, pressing intensity, possession style)
- Manager win rate in tournament knockouts

Model improvements

- Monte Carlo simulation (10,000 runs) for probabilistic bracket outcomes
- SHAP explainability - show which features drive each prediction
- Bayesian updating - as tournament progresses, update priors with evidence
- Neural network on sequence of recent results (LSTM/Transformer)
- Larger ensemble (add CatBoost, LightGBM, Neural Net)

12. Complete Reference List

Papers

Dixon & Coles (1997)

<https://doi.org/10.1111/1467-9876.00065>

Foundational paper for football score prediction with correlation correction.

Chen & Guestrin (2016) - XGBoost

<https://arxiv.org/abs/1603.02754>

The XGBoost algorithm paper - gradient boosted trees with regularization.

Breiman (2001) - Random Forests

<https://link.springer.com/article/10.1023/A:1010933404324>

Original Random Forest paper - ensemble of independent decision trees.

Elo (1978) - The Rating of Chess Players

https://en.wikipedia.org/wiki/Elo_rating_system

Foundation of the Elo rating system adapted for football.

Hvattum & Arntzen (2010) - ELO ratings for football

<https://doi.org/10.1016/j.ijforecast.2009.10.002>

Using Elo ratings as match predictors in football forecasting.

Tutorials & Documentation

XGBoost Documentation

<https://xgboost.readthedocs.io/en/latest/>

Official XGBoost docs - parameters, API, tutorials.

scikit-learn User Guide

https://scikit-learn.org/stable/user_guide.html

Logistic Regression, Random Forest, model evaluation metrics.

Dixon-Coles Python Implementation

<https://dashee87.github.io/football/python/predicting-football-results-with-statistical-modelling-dixon-coles/>

Step-by-step Python tutorial for implementing Dixon-Coles from scratch.

Poisson Distribution in Football

<https://www.pinnacle.com/en/betting-articles/Soccer/how-to-calculate-poisson-distribution/MD62MLXN3KR2GRA2>

Practical guide to using Poisson for football betting/prediction.

World Football Elo Ratings

<https://www.eloratings.net/about>

How eloratings.net computes and uses football Elo ratings.

Ensemble Learning Guide (MLWave)

<https://mlwave.com/kaggle-ensembling-guide/>

Comprehensive guide to stacking, blending, and ensemble techniques.

Data Sources

International Football Results

https://github.com/martj42/international_results

FIFA World Cup 2026 Prediction System - Code & ML Walkthrough

49,477 matches from 1872-2026. CC0 license. Primary training dataset.

World Cup Historical Database

<https://github.com/jfjelstul/worldcup>

1,248 World Cup matches with squads, goals, standings. 1930-2022.

Flag CDN

<https://flagcdn.com>

Free flag images supporting ISO codes including UK subdivisions (gb-eng, gb-sct).